类 C 语言词法分析器（Scanner）实现

姓名：李祥熙　学号：2234412799

2026 年 4 月 24 日

1　实验目的

本实验围绕“类 C 语言词法分析器”展开，目标如下：

- 根据给定词法规则，构建可执行的 DFA 思路并实现扫描器程序。
- 将源程序字符流切分为 token 序列，识别关键字、标识符、常数、运算符与界符。
- 将词法分析结果同时输出到终端与文件，作为后续语法分析阶段输入。

2　实验过程

2.1　实验环境

- 操作系统：macOS
- 编程语言：C（C11）
- 编译工具：GCC

2.2　词法规则与输入形式

本实验依据实验说明中的类 C 语言词法记号集合进行实现。核心规则如下：

- 标识符：以字母或下划线开头，后续可接字母、数字、下划线。
- 整数与实数：支持无符号/带符号形式，其中实数支持小数点形式。
- 关键字：`int`、`float`、`void`、`if`、`else`、`while`、`return`、`input`、`print`。
- 关系运算符与赋值：`<`，`<=`，`>`，`>=`，`==`，`!=`，`=`。
- 界符：`(` `)` `[` `]` `{` `}` `,` `;`。

2.3 实现步骤

1. 设计扫描器状态推进逻辑，维护当前字符、行列位置与前一 token 类型。
2. 实现空白符与注释跳过（`//` 单行注释、`/* */` 多行注释）。
3. 实现识别函数：标识符/关键字识别、数字识别、运算符与界符识别。
4. 统一输出格式为 `(TOKEN, lexeme)`，并将结果写入 `tokens.out`。
5. 在测试文件上运行，检查 token 序列正确性及错误统计信息。

3 实验内容

3.1 任务一：DFA 风格扫描器实现

本任务使用 C 语言实现一个顺序扫描器。核心思想是“读取当前字符并转移到对应识别分支”，每次成功识别一个最长 token 后输出。

其中，数字处理支持以下情况：

- 整数：如 `10`、`-3`。
- 浮点数：如 `3.14`、`-0.5`、`.75`。

为支持一元正负号与二元加减号区分，程序根据前一 token 是否为“值 token”判断当前 `+/-` 是否可并入数字。

3.2 任务二：token 输出与错误处理

识别到合法 token 后，输出到标准输出与结果文件。非法字符输出词法错误信息（带行列号）并继续扫描，最终给出错误总数。

本实验输出格式为：

$$\texttt{(TOKEN_NAME, lexeme)} \tag{1}$$

例如：`(VOID, void)`、`(ID, qsort)`、`(ROP, <=)`。

4 实验结果

4.1 扫描结果截图

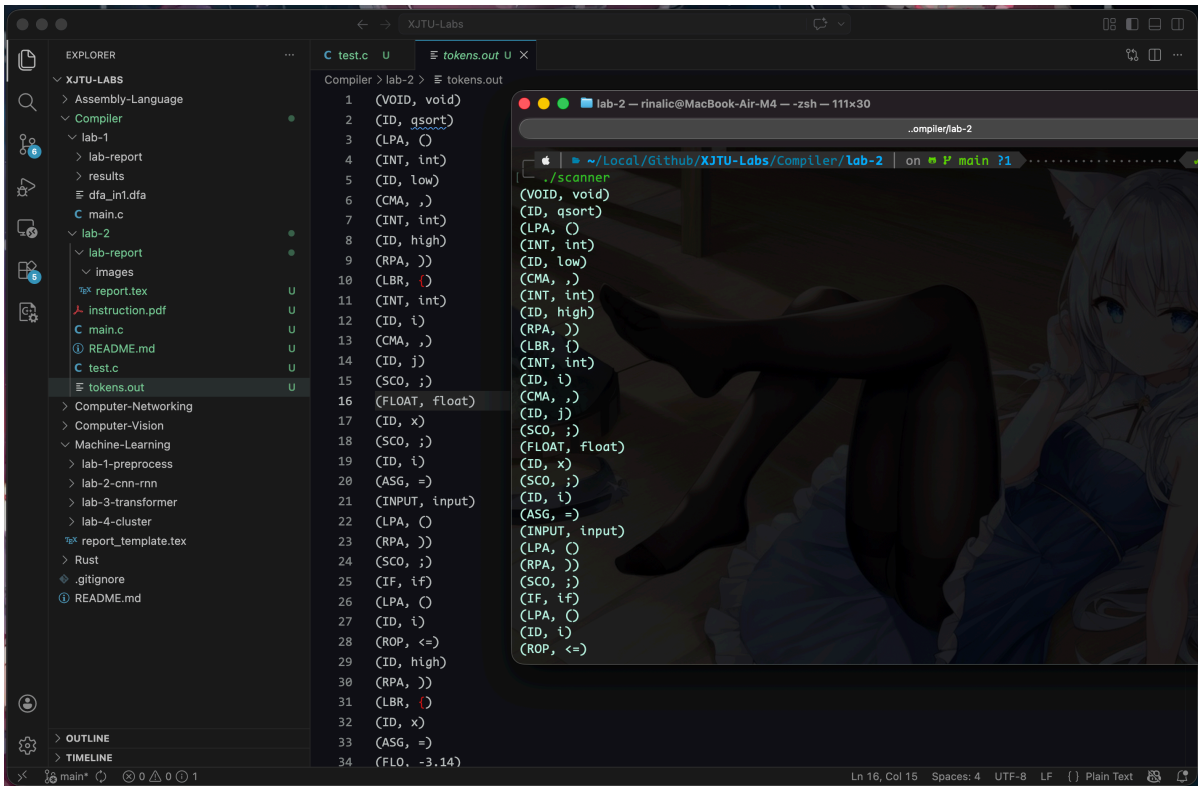

图 1: 默认测试文件运行结果（终端 token 输出）

结果分析:

- 关键字、标识符、运算符、界符可被正确区分并按顺序输出。
- 浮点数 `-3.14` 被识别为 `FLO`，符合预期。
- 结果文件末尾显示 `Scanner finished with 0 lexical error(s).`，说明测试样例下扫描成功。

4.2 代表性 token 统计

表 1: 样例程序中部分 token 统计

token 类型	出现次数	示例
ID	11	`qsort, i, high`
INT	4	`int, 10, 1`
FLO	1	`-3.14`
ROP	2	`<=, <`
结构界符	15	`( ) { } ; ,`

分析：该统计反映了样例程序包含函数定义、分支与循环结构，能够覆盖多数基础词法类型。

5 总结

本实验完成了类 C 语言 scanner 的设计与实现，达成了实验要求：

- 基于 DFA 思路实现了可运行的词法分析器。
- 实现了关键 token 的识别与规范化输出。
- 实现了终端与文件双输出，可直接衔接后续语法分析阶段。

后续可改进方向：

- 扩展字符串常量、字符常量等词法单元。
- 增加更细粒度错误恢复策略与错误分类。
- 将词法规则表格化，支持可配置词法定义。

附录：实验源代码

A.1 主要源码：main.c

```
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_LEXEME 256
```

```c
typedef enum
{
  TK_ID,
  TK_INT,
  TK_FLO,
  TK_ADD,
  TK_SUB,
  TK_MUL,
  TK_DIV,
  TK_ROP,
  TK_ASG,
  TK_LPA,
  TK_RPA,
  TK_LBK,
  TK_RBK,
  TK_LBR,
  TK_RBR,
  TK_CMA,
  TK_SCO,
  TK_IF,
  TK_ELSE,
  TK_WHILE,
  TK_RETURN,
  TK_INPUT,
  TK_PRINT,
  TK_VOID,
  TK_FLOAT_KW,
  TK_UNKNOWN
} TokenType;

typedef struct
{
  TokenType type;
  char lexeme[MAX_LEXEME];
  int line;
  int col;
} Token;

typedef struct
{
```

```
  FILE *fp;
  int line;
  int col;
  int ch;
} Scanner;

static const char *
token_name (TokenType t)
{
  switch (t)
    {
    case TK_ID:
      return "ID";
    case TK_INT:
      return "INT";
    case TK_FLO:
      return "FLO";
    case TK_ADD:
      return "ADD";
    case TK_SUB:
      return "SUB";
    case TK_MUL:
      return "MUL";
    case TK_DIV:
      return "DIV";
    case TK_ROP:
      return "ROP";
    case TK_ASG:
      return "ASG";
    case TK_LPA:
      return "LPA";
    case TK_RPA:
      return "RPA";
    case TK_LBK:
      return "LBK";
    case TK_RBK:
      return "RBK";
    case TK_LBR:
      return "LBR";
    case TK_RBR:
      return "RBR";
```

```
    case TK_CMA:
      return "CMA";
    case TK_SCO:
      return "SCO";
    case TK_IF:
      return "IF";
    case TK_ELSE:
      return "ELSE";
    case TK_WHILE:
      return "WHILE";
    case TK_RETURN:
      return "RETURN";
    case TK_INPUT:
      return "INPUT";
    case TK_PRINT:
      return "PRINT";
    case TK_VOID:
      return "VOID";
    case TK_FLOAT_KW:
      return "FLOAT";
    default:
      return "UNKNOWN";
    }
}

static int
is_value_token (TokenType t)
{
  return t == TK_ID || t == TK_INT || t == TK_FLO || t == TK_RPA || t
     == TK_RBK
         || t == TK_RBR;
}

static void
scanner_init (Scanner *s, FILE *fp)
{
  s->fp = fp;
  s->line = 1;
  s->col = 0;
  s->ch = fgetc (fp);
}
```

```
static void
scanner_advance (Scanner *s)
{
  if (s->ch == '\n')
    {
      s->line++;
      s->col = 0;
    }
  else
    {
      s->col++;
    }
  s->ch = fgetc (s->fp);
}

static int
peek_char (Scanner *s)
{
  int c = fgetc (s->fp);
  if (c != EOF)
    {
      ungetc (c, s->fp);
    }
  return c;
}

static void
skip_blanks_and_comments (Scanner *s)
{
  while (s->ch != EOF)
    {
      if (isspace ((unsigned char)s->ch))
        {
          scanner_advance (s);
          continue;
        }

      if (s->ch == '/')
        {
          int nxt = peek_char (s);
```

```
          if (nxt == '/')
            {
              scanner_advance (s);
              scanner_advance (s);
              while (s->ch != EOF && s->ch != '\n')
                {
                  scanner_advance (s);
                }
              continue;
            }
          if (nxt == '*')
            {
              scanner_advance (s);
              scanner_advance (s);
              while (s->ch != EOF)
                {
                  if (s->ch == '*')
                    {
                      scanner_advance (s);
                      if (s->ch == '/')
                        {
                          scanner_advance (s);
                          break;
                        }
                    }
                  else
                    {
                      scanner_advance (s);
                    }
                }
              continue;
            }
        }

      break;
    }
}

static TokenType
keyword_or_id (const char *s)
{
```

```
  if (strcmp (s, "int") == 0)
    {
      return TK_INT;
    }
  if (strcmp (s, "float") == 0)
    {
      return TK_FLOAT_KW;
    }
  if (strcmp (s, "void") == 0)
    {
      return TK_VOID;
    }
  if (strcmp (s, "if") == 0)
    {
      return TK_IF;
    }
  if (strcmp (s, "else") == 0)
    {
      return TK_ELSE;
    }
  if (strcmp (s, "while") == 0)
    {
      return TK_WHILE;
    }
  if (strcmp (s, "return") == 0)
    {
      return TK_RETURN;
    }
  if (strcmp (s, "input") == 0)
    {
      return TK_INPUT;
    }
  if (strcmp (s, "print") == 0)
    {
      return TK_PRINT;
    }
  return TK_ID;
}

static Token
make_simple (Scanner *s, TokenType type, char c)
```

```
{
  Token t;
  t.type = type;
  t.line = s->line;
  t.col = s->col + 1;
  t.lexeme[0] = c;
  t.lexeme[1] = '\0';
  scanner_advance (s);
  return t;
}

static Token
scan_identifier (Scanner *s)
{
  Token t;
  int i = 0;
  t.line = s->line;
  t.col = s->col + 1;

  while (s->ch != EOF
         && (isalpha ((unsigned char)s->ch) || isdigit ((unsigned char)
            s->ch)
             || s->ch == '_'))
    {
      if (i < MAX_LEXEME - 1)
        {
          t.lexeme[i++] = (char)s->ch;
        }
      scanner_advance (s);
    }
  t.lexeme[i] = '\0';
  t.type = keyword_or_id (t.lexeme);
  return t;
}

static Token
scan_number (Scanner *s, int allow_sign)
{
  Token t;
  int i = 0;
  int has_dot = 0;
```

```
int digits_before_dot = 0;
int digits_after_dot = 0;

t.line = s->line;
t.col = s->col + 1;

if (allow_sign && (s->ch == '+' || s->ch == '-'))
  {
    t.lexeme[i++] = (char)s->ch;
    scanner_advance (s);
  }

while (s->ch != EOF && isdigit ((unsigned char)s->ch))
  {
    if (i < MAX_LEXEME - 1)
      {
        t.lexeme[i++] = (char)s->ch;
      }
    digits_before_dot++;
    scanner_advance (s);
  }

if (s->ch == '.')
  {
    has_dot = 1;
    if (i < MAX_LEXEME - 1)
      {
        t.lexeme[i++] = '.';
      }
    scanner_advance (s);

    while (s->ch != EOF && isdigit ((unsigned char)s->ch))
      {
        if (i < MAX_LEXEME - 1)
          {
            t.lexeme[i++] = (char)s->ch;
          }
        digits_after_dot++;
        scanner_advance (s);
      }
  }
```

```
  t.lexeme[i] = '\0';

  if (has_dot && (digits_before_dot > 0 || digits_after_dot > 0))
    {
      t.type = TK_FLO;
    }
  else
    {
      t.type = TK_INT;
    }

  return t;
}

static Token
scan_next_token (Scanner *s, TokenType prev)
{
  skip_blanks_and_comments (s);

  if (s->ch == EOF)
    {
      Token end;
      end.type = TK_UNKNOWN;
      end.lexeme[0] = '\0';
      end.line = s->line;
      end.col = s->col + 1;
      return end;
    }

  if (isalpha ((unsigned char)s->ch) || s->ch == '_')
    {
      return scan_identifier (s);
    }

  if (isdigit ((unsigned char)s->ch)
      || ((s->ch == '+' || s->ch == '-') && !is_value_token (prev)
          && (isdigit ((unsigned char)peek_char (s)) || peek_char (s)
              == '.'))
      || (s->ch == '.' && isdigit ((unsigned char)peek_char (s))))
    {
```

```
      int allow_sign = (s->ch == '+' || s->ch == '-');
      return scan_number (s, allow_sign);
    }

  if (s->ch == '<' || s->ch == '>' || s->ch == '=' || s->ch == '!')
    {
      Token t;
      t.line = s->line;
      t.col = s->col + 1;
      t.lexeme[0] = (char)s->ch;
      t.lexeme[1] = '\0';

      if (s->ch == '=')
        {
          scanner_advance (s);
          if (s->ch == '=')
            {
              t.type = TK_ROP;
              t.lexeme[1] = '=';
              t.lexeme[2] = '\0';
              scanner_advance (s);
            }
          else
            {
              t.type = TK_ASG;
            }
          return t;
        }

      scanner_advance (s);
      if (s->ch == '=')
        {
          t.lexeme[1] = '=';
          t.lexeme[2] = '\0';
          scanner_advance (s);
        }
      t.type = TK_ROP;
      return t;
    }

  if (s->ch == '+')
```

```
    {
      return make_simple (s, TK_ADD, '+');
    }
  if (s->ch == '-')
    {
      return make_simple (s, TK_SUB, '-');
    }
  if (s->ch == '*')
    {
      return make_simple (s, TK_MUL, '*');
    }
  if (s->ch == '/')
    {
      return make_simple (s, TK_DIV, '/');
    }
  if (s->ch == '(')
    {
      return make_simple (s, TK_LPA, '(');
    }
  if (s->ch == ')')
    {
      return make_simple (s, TK_RPA, ')');
    }
  if (s->ch == '[')
    {
      return make_simple (s, TK_LBK, '[');
    }
  if (s->ch == ']')
    {
      return make_simple (s, TK_RBK, ']');
    }
  if (s->ch == '{')
    {
      return make_simple (s, TK_LBR, '{');
    }
  if (s->ch == '}')
    {
      return make_simple (s, TK_RBR, '}');
    }
  if (s->ch == ',')
    {
```

```
      return make_simple (s, TK_CMA, ',');
    }
  if (s->ch == ';')
    {
      return make_simple (s, TK_SCO, ';');
    }

  Token bad;
  bad.type = TK_UNKNOWN;
  bad.line = s->line;
  bad.col = s->col + 1;
  bad.lexeme[0] = (char)s->ch;
  bad.lexeme[1] = '\0';
  scanner_advance (s);
  return bad;
}

int
main (int argc, char **argv)
{
  const char *input_path = "test.c";
  const char *output_path = "tokens.out";
  if (argc >= 2)
    {
      input_path = argv[1];
    }
  if (argc >= 3)
    {
      output_path = argv[2];
    }

  FILE *in = fopen (input_path, "r");
  if (!in)
    {
      fprintf (stderr, "Cannot open input file: %s\n", input_path);
      return 1;
    }

  FILE *out = fopen (output_path, "w");
  if (!out)
    {
```

```
    fprintf (stderr, "Cannot open output file: %s\n", output_path);
    fclose (in);
    return 1;
  }

Scanner s;
scanner_init (&s, in);

TokenType prev = TK_UNKNOWN;
int error_count = 0;

while (1)
  {
    Token t = scan_next_token (&s, prev);
    if (t.type == TK_UNKNOWN && t.lexeme[0] == '\0')
      {
        break;
      }

    if (t.type == TK_UNKNOWN)
      {
        fprintf (stderr, "Lexical error at %d:%d, unexpected '%s'\n",
            t.line,
                 t.col, t.lexeme);
        error_count++;
        prev = TK_UNKNOWN;
        continue;
      }

    fprintf (stdout, "(%s, %s)\n", token_name (t.type), t.lexeme);
    fprintf (out, "(%s, %s)\n", token_name (t.type), t.lexeme);
    prev = t.type;
  }

fprintf (stdout, "Scanner finished with %d lexical error(s).\n",
         error_count);
fprintf (out, "Scanner finished with %d lexical error(s).\n",
   error_count);

fclose (out);
fclose (in);
```

```
  return 0;
}
```

A.2 测试输入：test.c

```
void
qsort (int low, int high)
{
  int i, j;
  float x;

  i = input ();
  if (i <= high)
    {
      x = -3.14;
      print (i);
    }
  else
    {
      while (j < 10)
        {
          j = j + 1;
        }
    }
  return;
}
```

A.3 运行命令

```
# 在 Compiler/lab-2 目录下
gcc -std=c11 -Wall -Wextra -O2 main.c -o scanner

# 默认读取 test.c, 输出到 tokens.out
./scanner

# 指定输入输出文件
./scanner test.c tokens.out
```